

Git & GitHub.

Day 1 we set up tools. Day 2 we wrote code worth saving. Today we learn to save it the way real teams do — under version control, on GitHub, with a habit of meaningful commits.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

3 of 24

How we'll spend our 90 minutes today.

90 min DURATION

More keyboard, fewer slides. Today is about building the Git habit — every command typed live, every student pushing to GitHub by the end.

00:00 – 00:10



Day 2 Recap & Q&A

Functions, files, the contact list cleaner. Quick check, then we move on.

00:10 – 00:25



Why Git Matters

The two-people-one-file problem. Why "Save As v2 final FINAL" is not version control.

00:25 – 00:45



Your First Repo + Commit

GitHub account, create a repo, git status/add/commit/push — typed live.

00:45 – 01:00



.gitignore + Pitfalls

What NOT to commit (.env, secrets, huge files) and five traps that bite beginners.

01:00 – 01:20



Merge Conflicts, Escalating

Two demos: different lines (easy auto-merge fail), then same line (true conflict, manual resolve).

01:20 – 01:30



Push Day 2 Work + Close

Each student pushes clean_contacts.py to their own repo. Day 4 preview.

Yesterday you wrote code worth saving. Today we save it.

Quick check that your Day 2 work is on your laptop and ready to commit. If it's not, we fix it now — today's exercise depends on it.

WHAT WE COVERED YESTERDAY

- ✓ Functions with real inputs and outputs — return, not print
- ✓ Reading files three ways — read(), readlines(), line-by-line
- ✓ Writing files — 'r' / 'w' / 'a' modes and the destructive nature of 'w'
- ✓ try/except for FileNotFoundError — graceful failure instead of crashes
- ✓ Built clean_contacts() — reads a messy contacts.txt, returns (name, email) tuples

CHECK YOUR LAPTOP NOW

```
# You should have these files from Day 2:
$ ls day2/ clean_contacts.py contacts.txt
# The function should still work: $
python clean_contacts.py ('Aarav',
'aarav@example.com') ('Priya',
'priya@example.com') ... # Git should be
installed (from Day 1): $ git --version
git version 2.43.0
```

⚠ **IF clean_contacts.py IS MISSING** — pair up with a neighbour, copy their file, we'll commit it today. **IF git --version fails** — raise your hand, we reinstall now.

Two people. One file. Three weeks of work. What could go wrong?

Git exists because of a problem every team hits. Let's walk through it.

WITHOUT GIT THE HORROR STORY

- 01 Aarav and Priya both clone the project folder from a shared drive on Monday.
- 02 Aarav edits `clean_contacts.py` on Tuesday. Priya edits the same file on Wednesday.
- 03 Aarav emails his version to Priya on Thursday. Priya's changes overwrite Aarav's. *Aarav's work is gone.*
- 04 Friday: they try to merge manually. Two hours later, they're not sure which version is correct.
- 05 Monday: they rename the file `clean_contacts_FINAL_v3_really_final.py`. This is not version control.

Every team that doesn't use Git lives this story. Every team.

WITH GIT THE SAME WEEK

- 01 Aarav and Priya both clone the same GitHub repo on Monday. Same starting point.
- 02 Aarav edits `clean_contacts.py` on Tuesday. Commits with a message: "Add email validation."
- 03 Priya edits the same file on Wednesday. Commits with a message: "Skip lines without name."
- 04 Thursday: Priya pulls Aarav's commit. Git merges both changes automatically. *Nothing is lost.*
- 05 Friday: if they edited the same line, Git flags it as a conflict. They resolve it in 2 minutes, not 2 hours.

Same work. Zero lost changes. A history of every decision, forever.

KEY IDEA Git doesn't just save your code. It saves **every version of your code**, with a message explaining why each change happened. That history is the real product.

Git is the tool. GitHub is the place. They are not the same thing.

You can use Git without GitHub. You cannot use GitHub without Git. Let's get this straight before we type any commands.

Git

THE TOOL - RUNS ON YOUR LAPTOP

Git is a command-line tool installed on your laptop. It tracks changes to files in a folder.

It works completely offline. You can commit, branch, and merge without any internet connection.

Git was built by **Linus Torvalds** in **2005** — the same person who built Linux. It's free, open-source, and runs every major tech company's codebase.

```
$ git --version
git version 2.43.0
```

GitHub

THE PLACE - RUNS IN THE CLOUD

GitHub is a website (github.com) that hosts your Git repositories online. Think of it as **Google Drive for code**, but built around Git.

It adds collaboration features Git doesn't have: pull requests, issues, code review, project boards.

GitHub was bought by **Microsoft** in **2018**. It's free for public repos and small private ones. There are alternatives (GitLab, Bitbucket) but GitHub is the industry default.

```
$ git push origin main # → sends your commits to GitHub
```



local

git commit



.git/

git push



github.com

git pull



local

Your first GitHub repo. Five minutes, start to finish.

✓ WHAT TO DO

- 1 Create your GitHub account
Go to `github.com` → Sign up. Use the SAME email you configured in Git on Day 1 (`git config --global user.email`). This matters — if they don't match, your commits won't show on your GitHub profile.
- 2 Create a new repository
Click the `+` icon in the top-right → New repository. Name it `day3-practice`. Set it to Public (so we can see each other's). Tick "Add a README file". Click Create repository.
- 3 Copy the clone URL
Click the green "Code" button. Copy the HTTPS URL (looks like `https://github.com/yourname/day3-practice.git`). Don't use SSH — we'll cover that later.
- 4 Clone it to your laptop
In your terminal, navigate to where you want the folder. Run `git clone <paste-url-here>`. This creates a `day3-practice/` folder with the README inside.
- 5 Open it in VS Code
`code day3-practice` (or open VS Code → File → Open Folder). You should see `README.md` inside.

We do this together. Nobody moves on until every student has a repo with a green README on their GitHub profile.

✓ VERIFY IT WORKED

```
$ git clone https://github.com/yourname/day3-
practice.git
Cloning into 'day3-practice'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0),
pack-reused 0
Receiving objects: 100% (3/3), done.
$ cd day3-practice
$ ls
README.md
$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

 **IF IT ASKS FOR A PASSWORD** — GitHub no longer accepts account passwords for Git. Use a Personal Access Token (we'll cover this on the next slide if anyone hits it). For now, raise your hand.

Four commands. That's 90% of Git.

Every Git session, for the rest of your career, will use these four commands in this order. Learn the loop.



```

# 1. Check what changed
$ git status
Changes not staged for commit:
modified:   clean_contacts.py
# 2. Stage the changes
$ git add clean_contacts.py
# or stage everything: $ git add .
# 3. Commit with a message
$ git commit -m "Add email validation to clean_contacts"
[main 7a3f2b1] Add email validation to clean_contacts
1 file changed, 5 insertions(+)
# 4. Push to GitHub
$ git push origin main
Enumerating objects: 5, done.
To github.com:yourname/day3-practice.git
5b2c1a0..7a3f2b1  main -> main
  
```

KEY IDEA A good commit message answers "WHY did I change this?" not "WHAT did I change?" The diff already shows the what.

We do this together. Every student, one commit, pushed to GitHub.

Open your `day3-practice` folder in VS Code. Follow along. By the end of this slide, your GitHub repo has a `green commit`.

```

①# In VS Code, create a new file called notes.md
# Add one line: "My first commit on Day 3."
# Save the file.
②$ git status
Untracked files:
notes.md
③$ git add notes.md
$ git status
Changes to be committed:
new file:   notes.md
④$ git commit -m "Add Day 3 notes file"
[main 8c4d9e2] Add Day 3 notes file
1 file changed, 1 insertion(+)
create mode 100644 notes.md
⑤$ git push origin main
Enumerating objects: 4, done.
To github.com:yourname/day3-practice.git
7a3f2b1..8c4d9e2  main -> main
⑥# Now refresh your GitHub repo page in the browser.
# You should see notes.md with your commit message.
# That green square on your GitHub profile? That's this commit.

```

STEP BY STEP

- ① Create a file in VS Code
Just a text file called `notes.md` with one line. Nothing fancy. The point is the Git workflow, not the content.
- ② `git status` — check what changed
Git sees a new untracked file. Red means "not staged yet." Run `git status` CONSTANTLY — it tells you where you are.
- ③ `git add` — stage the file
Move `notes.md` into the staging area. Run `git status` again — now it's green, meaning "ready to commit."
- ④ `git commit -m` — save with a message
The `-m` flag lets you write the message inline. Keep it short, imperative, and descriptive: "Add X" not "added X."
- ⑤ `git push` — upload to GitHub
Your local commit is now on GitHub. Refresh the repo page in your browser to see it.
- ⑥ Verify on GitHub
The commit appears on your repo page. Your GitHub profile gets a **green contribution square** for today. This is the dopamine hit that builds the habit.

Not everything in your folder belongs on GitHub.

A `.gitignore` file tells Git which files to skip. Get this right on Day 1 of a project, not Day 30 when your repo is 500 MB of junk.

? WHY IGNORE?



Secrets and credentials

`.env` files, API keys, database passwords. Committing these is the **#1 way beginners leak secrets publicly**. Once it's on GitHub, assume it's compromised.



Huge files that bloat the repo

Datasets, model weights, videos, images. GitHub has a **100 MB file limit**. Use Git LFS or external storage for big assets.



Generated and machine-specific files

`__pycache__/, .venv/, .DS_Store, *.pyc`. These are recreated by your machine. Committing them causes conflicts and noise.



Personal notes and drafts

`TODO.md, scratch/, notes-local.md`. Keep these local. Your repo is for the project, not your journal.

PYTHON .GITIGNORE TEMPLATE

Copy this into your repo today

```
# .gitignore - Python project template

# --- IDE / editor ---
.idea/
*.swp
*.suo
*.DS_Store

# --- Data and assets (keep these out of Git) ---
data/
*.csv
*.json
*.xml
*.db
*.sqlite3

# --- Build and distribution ---
build/
dist/
*.egg-info/

# --- Virtual environments ---
.venv/

# --- Python cache and compiled ---
__pycache__/
*.pyc
*.pyo
*.pyd
*.pyc
*.pyo
*.pyd
*.so

# --- Environment and secrets ---
.env
*.env.local
secrets.json
*.key
```

IF YOU ALREADY COMMITTED SOMETHING YOU SHOULDN'T HAVE

`git rm --cached <file>` removes it from Git tracking without deleting it from your laptop. Then add it to `.gitignore` and commit.

Five Git traps every beginner falls into. Once.

You will hit these. Best to see them on a slide first, so you recognise them in your own work later.

01 COMMITTING .venv/ OR __pycache__

```
# x Bad — your repo is now 200 MB of junk
$ git add .
$ git commit -m "stuff"
$ git push
# → GitHub repo is huge, clones take forever,
# merge conflicts on .pyc files constantly
```

✗ Add .venv/ and __pycache__/ to .gitignore **BEFORE** your first commit.

02 COMMITTING SECRETS (.env, API KEYS)

```
# x Bad — your API key is now public
$ git add .env
$ git commit -m "add config"
$ git push
# → Anyone can read your .env on GitHub.
# Even if you delete it later, it's in the history.
```

🔑 Add .env to .gitignore. If already committed, **rotate the key immediately** — assume it's leaked.

03 git push --force ON A SHARED BRANCH

```
# x Bad — overwrites your teammate's commits
$ git push --force origin main
# → If a teammate pushed while you were working,
# their commits are GONE from GitHub.
# Force-push only on your own branches.
```

⚠ Never --force on main or shared branches. Use --force-with-lease if you must force-push.

04 BAD COMMIT MESSAGES

```
# x Bad — useless history
$ git commit -m "fix"
$ git commit -m "stuff"
$ git commit -m "asdfasdf"
$ git commit -m "final"
# ✓ Good — tells future-you what happened
$ git commit -m "Add email validation to clean_contacts"
$ git commit -m "Skip blank lines in contact parser"
```

📝 Imperative mood, under 50 chars, describes the WHY. "Add X" not "changed stuff."

05 git add . WITHOUT CHECKING STATUS FIRST

```
# x Bad — stages everything, including junk
$ git add .
$ git commit -m "update"
# → You just committed debug print statements,
# a 500MB data file, and your .env
# ✓ Good — check first, stage selectively
$ git status
$ git add clean_contacts.py
$ git commit -m "Fix tuple unpacking in clean_contacts"
```

🕒 Always git status before git add. Stage specific files, not git add .

★ KEY TAKEAWAY

Two habits prevent all five traps.

Every one of these is preventable with two habits:

- 1 Always check git status before staging.
- 2 Keep your .gitignore up to date.

02/03

When two people touch the same file.

This is the moment Git stops being magic and becomes a tool you actually understand. Two demos, escalating from easy to hard.



Two edits. Different lines. Git tries to auto-merge — and sometimes fails.

We set this up deliberately. Aarav edits line 3, Priya edits line 7. Same file, different parts. Watch what happens.

TERMINAL — DAY3-PRACTICE · ZSH

```
# SETUP - both students have the same README.md

# Line 1: Day 3 Practice

# Line 2: (blank)

# Line 3: Something here

# Line 4: (blank)

# Line 5: TODO

# Line 6: Add more notes

# Line 7: - Add contact info

# Aarav edits line 3, commits, pushes

# Line 3: Notes from Day 3 - Git & Github.

$
$ "Update notes line with day 3 topic"
$

# Priya edits line 7, commits, tries to push

# Line 7: - Add contact info (Priya's version)

$
$ "Mark contact info line as mine"
$

# ! [rejected] main -> main (Push forced)

# updates were rejected because the remote contains
# work that you do not have locally.

# Priya pulls first

$

# ...Auto-merging README.md
# Merge made by the 'ort' strategy.
# README.md
DAY 03: GIT & GITHUB
# 1 file changed, 1 insertion(+), 1 deletion(-)
```

WHAT JUST HAPPENED?

Auto-merge ✓

- 1 Aarav pushed first — his commit is on GitHub.
- 2 Priya tried to push — Git rejected because she's behind.
- 3 Priya pulled — Git auto-merged the two changes. Different lines = no conflict.
- 4 Priya pushed again — success. Both edits live on GitHub.

KEY TAKEAWAY

Different lines, same file = Git auto-merges. This is the happy path. `git pull` before `git push` is the habit that prevents most conflicts.

Two edits. Same line. Git cannot decide. You must.

This is a real merge conflict. Git marks the file with conflict markers. You edit the file, choose which version wins, then commit.

```

● ● ● TERMINAL + README.MD • ZSH

# DETAIL -- view README.md, line 3 (new)
# Line 3: Notes go here.

# AARAV changes line 3 in
# Notes from Day 3 -- Git & GitHub
# Commits and pushes.

# PRIYA changes line 3 in
# My personal Day 3 notes.
# Commits, tries to push, gets rejected
# Priya pulls:

# ...Auto-merging README.md
# ...CONFLICT (content): Merge conflict in README.md
# ...Automatic merge failed; fix conflicts and
#   commit the result.

# Priya opens README.md in VS Code. She sees:

<<<<<<< HEAD
=====
>>>>>> origin/main

# Show merge like this to keep BOTH (or push one)
# Line 3: Notes from Day 3 -- Git & GitHub. (Priya's annotation)

# Then she stages, commits, pushes:
DAY 03 • GIT & GITHUB
$
$ "Resolve conflict on notes line -- keep Aarav's version with annotation"
$

```

HOW TO READ CONFLICT MARKERS

4 markers

- 1 <<<<<<< HEAD — Start of YOUR local changes. Everything between this and ===== is what you had on your laptop.
- 2 ===== — The divider. Above is yours, below is theirs (what came from GitHub).
- 3 >>>>>>> origin/main — End of the incoming changes. Everything between ===== and this is what your teammate pushed.
- 4 **RESOLUTION** — Edit the file: delete the markers, keep what you want (both versions, one version, or a merged version). Then `git add + git commit + git push`.

VS CODE HELPS

When Git detects a conflict, VS Code shows **Accept Current**, **Accept Incoming**, **Accept Both** buttons above the conflict. Use them. You don't have to edit markers manually.

Today we worked on **main**. Real teams don't.

Branches let you build a feature without breaking the main code. We won't go deep today — just enough to recognise what you'll see in Phase 4.

THE MENTAL MODEL

DIAGRAM

```
# main is the "stable" branch — always works
main:  A  B  C  D  M (merge)

# Feature branch: /
feature:      X  Y  Z

# X, Y, Z are commits on a branch.
# They don't affect main until you merge.
# If the feature breaks, you just abandon the branch.
# main stays clean.
```



`git checkout -b feature-name`

Creates a new branch and switches to it. Your changes now live on this branch, not main.



`git merge feature-name`

Brings the branch's changes back into main. This is where conflicts can happen.



`git branch -d feature-name`

Deletes the branch after merging. Clean-up. The commits stay on main.

WHY YOU'LL CARE IN PHASE 4

DAYS 15-21

- 1 In **Phase 4 (Days 15-21)**, your group will work on a project together. If everyone commits to **main**, you'll constantly break each other's code.
- 2 The pattern: each group member creates a branch for their feature (e.g. **add-email-validation**, **fix-data-loading**). They build on their branch, test it, then merge to main.
- 3 This is called **Feature Branch Workflow**. It's how every real software team works. We'll practice it from Day 15 onward.
- 4 For today and the next 11 days, you'll work on **main**. That's fine — you're solo. Branching matters when there are teammates.



TODAY'S TAKEAWAY

Branches exist. You'll use them in Phase 4. For now, **main** is all you need.

Push your Day 2 work to your own GitHub repo.

This is today's gate. Every student leaves with a green contribution square and a real Python file on GitHub — proof you wrote code worth saving.

THE TASK

SOLO

BRIEF

Take your `clean_contacts.py` and `contacts.txt` from Day 2. Put them in your `day3-practice` repo. Commit them with a meaningful message. Push to GitHub. Verify the files appear on your repo page.

REQUIREMENTS

- ✓ Both files (`clean_contacts.py` + `contacts.txt`) are in the repo
- ✓ A `.gitignore` file exists (use the template from slide 9 — at minimum ignore `.venv/` and `__pycache__`)
- ✓ Commit message is descriptive (NOT "stuff" or "update")
- ✓ Files are visible on `github.com/yourname/day3-practice`
- ✓ You can clone your own repo to a fresh folder and run `python clean_contacts.py` successfully

✓ BY THE END

Every student has pushed at least one commit to their own GitHub repo. That's today's checkpoint — and the Session Plan's stated goal for **Day 3**.

EXPECTED FLOW

6 STEPS

TERMINAL — DAY3-PRACTICE · ZSH

```
# 1. Copy Day 2 files into your repo
$
$

# 2. Create a .gitignore (use slide 9 template)
$
# → paste the Python template, save

# 3. Check what's about to be committed
$
# ... should see: clean_contacts.py, contacts.txt
# .gitignore as untracked files

# 4. Stage and commit
$
$ "Add Day 2 contact list cleaner exercise"

# 5. Push to GitHub
$

# 6. Verify → open github.com/yourname/day3-practice
# → you should see all 3 files

# → your GitHub profile has a green square for today
```

Phase 1 is done. You have tools, code, and version control.

Tomorrow we start **Phase 2 — Building Interfaces in Streamlit**. Your Python becomes an app people can click.

OPEN THE FLOOR

Ask Now

- ✓ "Which Git command still feels unclear — `status`, `add`, `commit`, `push`, or `pull`?"
- ✓ "Did the merge conflict demo click, or does it feel scary?"
- ✓ "Anything from the pitfalls slide you've already hit?"

BETWEEN SESSIONS

Get Unstuck

- ✓ Email · hello@zlaark.com
For Git questions, include the exact command and error message.
- ✓ Web · www.zlaark.com
Program updates and resources.
- ✓ In-session · raise a hand
No question is too small. We move at the room's pace.

WHAT'S NEXT · PHASE 2 Days 4–8

- DAY 4** **Meet Streamlit.** Turn a Python script into a browser app.
- DAYS 5–7** **Widgets, layouts, charts.** Build a real dashboard.
- DAY 8** **Solo challenge.** The gate before group formation.